

包丁検出・刃渡り計測アプリ

技術解説 — 計算原理と数式

本書は、ブラウザ上で動作する包丁計測アプリの主要アルゴリズムを、数式・図解を交えて説明する技術ドキュメントです。扱うトピックは「画像処理による包丁検出」「刃渡り推定」「刃渡り曲線抽出」「V字断面生成」「3D合成・アライメント」「CSV出力」の6つです。

1. 画像処理・包丁検出 (OpenCV.js)

1-1. 前処理パイプライン

カメラ画像に対して以下の順でOpenCV.jsの処理を適用します。

① グレースケール変換

RGBカラー画像をグレースケールに変換し、処理量を削減します。

$$I_{\text{gray}}(x,y) = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

② ガウシアンブラー(ノイズ除去)

カーネルサイズ $(2k+1) \times (2k+1)$ のガウシアンフィルタで高周波ノイズを除去します。

$$G(x,y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2+y^2) / 2\sigma^2)$$

※ アプリのデフォルト: カーネルサイズ 5×5、 σ 自動 (OpenCV標準)

③ Cannyエッジ検出

ガウシアン平滑化後の画像に Canny アルゴリズムを適用し、包丁の輪郭エッジを抽出します。Canny は以下の4ステップで構成されます。

- ・ Sobelフィルタで勾配強度 $|\nabla I|$ と方向 θ を計算
- ・ Non-maximum suppression (エッジ方向の極大のみ残す)
- ・ ヒステリシス閾値処理: 強エッジ (threshold2以上) から弱エッジ (threshold1以上) を追跡

1-2. 輪郭検出とスコアリング

Cannyエッジ画像から `findContours` で輪郭を抽出し、以下のスコアで包丁候補を選びます。

$$\text{score} = \text{area} \times \min(\text{aspect}, 15)$$

ここで $\text{aspect} = w/h$ (最小外接矩形の長辺/短辺) です。aspect が 2.5 ~ 25 の範囲外の輪郭は除外します (極端に丸い・細すぎる形状を排除)。最大スコアの輪郭が包丁候補として採用されます。

1-3. 最小外接矩形 (minAreaRect)

採用された輪郭に対し、面積最小の回転外接矩形を求めます。OpenCV の `minAreaRect` はモーメント行列の固有値分解を使い、以下を返します。

- ・ center: 矩形の重心 (cx, cy)
- ・ size: (width, height)
- ・ angle: X軸から width ベクトルへの角度 (-90° , 0°]

幅と高さのうち長い方が包丁の全長方向、短い方が幅方向です。

2. 刃渡り推定(幅プロファイル解析)

2-1. 座標回転

最小外接矩形の角度 α だけ輪郭点群を回転させ、長軸が水平 (x軸平行) になるよう揃えます。

$$x' = \cos \alpha \cdot (px - cx) + \sin \alpha \cdot (py - cy)$$

$$y' = -\sin \alpha \cdot (px - cx) + \cos \alpha \cdot (py - cy)$$

ただし OpenCV の angle 定義では height > width のとき +90° 補正が必要です。

2-2. 幅プロファイルの構築

回転後の点群を x 軸方向に 50 ビン (等間隔) で分割し、各ビン b における幅 W(b) を求めます。

$$\text{binSize} = (x_{\text{max}} - x_{\text{min}}) / 50$$

$$b = \text{floor}((x' - x_{\text{min}}) / \text{binSize}), b \in [0, 49]$$

$$W(b) = \max Y(b) - \min Y(b)$$

$\max Y(b)$, $\min Y(b)$ はビン b に属する全点の回転後 y' の最大・最小値です。

2-3. 移動平均スムージング

幅プロファイルに窓幅 3 の移動平均を適用してノイズを除去します。

$$W_{\text{smooth}}(b) = (1/|N_b|) \cdot \sum_{j \in N_b} W(j), N_b = \{b-1, b, b+1\}$$

※ $W(j)=0$ のビンは平均計算から除外します。

2-4. 刃先側の判定

幅プロファイルの最大値位置 b_{max} を求め、刃先の方を判定します。包丁は「刃先 (細) → 刃元 (最大幅) → 柄」の順に幅が変化するため、最大幅位置が中央より左にあれば刃先は右、右にあれば刃先は左です。

$$\text{tipSide} = \text{"left"} \text{ if } b_{\text{max}} \leq 25 \text{ else "right"}$$

2-5. アゴ位置の検出 (50%閾値法)

アゴとは刃部分と柄部分の境界点です。以下の手順で検出します。

- ・ 柄端ゾーン (刃先から遠い20%の範囲) の最小 y' を柄基準線 Y_{handle} とする
- ・ 刃元ビン b_{max} の底面 y' を刃基準線 Y_{heel} とする
- ・ 段差 $\Delta Y = Y_{\text{heel}} - Y_{\text{handle}}$ を計算

$$\text{threshold} = Y_{\text{handle}} + 0.50 \times \Delta Y$$

柄端側から刃先方向にスキャンし、底面 y' が threshold を初めて超えたビンをアゴ位置 b_{junc} とします。

2-6. 刃渡り(px → mm 変換)

$$L_{\text{px}} = |b_{\text{junc}} - b_{\text{tip}}| \times \text{binSize}$$

$$L_{\text{mm}} = L_{\text{px}} / \text{PPM}$$

PPM (pixels per mm) は後述のカード校正で求めます。

3. カード校正 (px/mm 変換係数の算出)

クレジットカード (ISO/IEC 7810 ID-1) の短辺 53.98 mm、またはA4用紙の短辺 210 mm を基準として、1mm あたりのピクセル数 (PPM) を算出します。

3-1. 3点からの垂線距離

エッジ画像上で短辺の2端点 P_1 , P_2 と反対側短辺の任意点 P_3 を指定します。 P_3 から直線 P_1P_2 への垂足 F を求め、距離 d を校正基準とします。

$$d = P_1P_2 \times P_1P_3 \text{ 方向ベクトルに対する } P_3 \text{ の垂線距離}$$

具体的には以下の垂足公式を使います。

$$t = ((P_3 - P_1) \cdot (P_2 - P_1)) / |P_2 - P_1|^2$$

$$F = P_1 + t \cdot (P_2 - P_1)$$

$$d = |P_3 - F| \text{ [px]}$$

3-2. PPM の算出

$$\text{PPM} = d_{\text{px}} / d_{\text{mm}}$$

ここで d_{mm} はカード短辺の実寸法です。以降の全ての mm 換算にこの PPM を使います。

4. 刃渡り曲線抽出

4-1. アゴ・切先の指定とスナップ

ユーザーがエッジ画像上でアゴと切先をタップすると、半径 r 内で最も明るいエッジピクセルに自動スナップします。

```
snap(x,y) = argmin_{(ix,iy) : dist(x,y, ix,iy) < r, brightness>64} dist(x,y, ix,iy)
```

4-2. エッジピクセルのトレース

アゴ (x_a, y_a) から切先 (x_c, y_c) まで直線を等分し、各ステップ s で垂直方向にウィンドウ $W=50px$ をスキャンして最近傍エッジ点を取得します。

```
t_s = s / steps, s = 0, 1, ..., steps
```

```
ex_s = round(x_a + (x_c - x_a) * t_s)
```

```
ey_s = argmin_{r \in [-W, W]} { r : isEdge(ex_s, ey_s + r) }
```

※ isEdge: ピクセル輝度が64超かどうかで判定します。

4-3. 曲線長の計算(弧長積分の離散近似)

取得した全トレース点 $\{(imgX_i, imgY_i)\}$ から弧長を計算します。

$$L = (1/PPM) \times \sum_{i=1}^{N-1} \sqrt{(imgX_i - imgX_{i-1})^2 + (imgY_i - imgY_{i-1})^2}$$

4-4. 等間隔サンプリング(ドット間隔)

曲線を指定間隔（デフォルト 10mm）でサンプリングします。トレース点は xMm 属性（アゴからの累積距離）を持ち、 xMm が前回サンプル点から $intervalMm$ 以上離れた点を採用します。

```
S = { P_i : xMm(P_i) - xMm(P_{last}) > intervalMm }
```

5. V字断面生成(刃先形状タブ)

包丁の刃は断面がV字形をしています。ユーザーが左右の θ 角（刃角）を指定すると、断面の3D形状を生成します。

5-1. セグメント方式

V字断面を n セグメントに分割し、各セグメントで独立した θ 角を設定できます。Z方向（深さ）を n 等分したとき、セグメント k の刃角を θ_k とすると、セグメント境界点の x 座標（側面の広がり）は以下で計算されます。

$$dz = Z_{\text{total}} / n$$

$$x_k = \sum_{j=0}^{k-1} dz \cdot \tan(\theta_j) \quad (k = 1, \dots, n)$$

これは刃先 ($x=0$) から外側へ向かう累積展開量です。左右で独立に計算します。

5-2. 断面の全幅

$$W_{\text{total}} = x_L(n) + x_R(n)$$

ここで $x_L(n)$, $x_R(n)$ は左右の n 番目の境界点の x 座標です。

5-3. 断面の繰り返し配置

刃渡り曲線上のサンプル点ごとに上記断面をY方向（刃渡り方向）に配置します。各断面の z の原点は刃先（V字の谷）、 y は刃渡り方向の位置です。

※ Y方向の間隔 $yStep$ はドット間隔 $intervalMm$ と自動同期します。

6. 3D合成・アライメント(刃先形状 × 刃渡り曲線)

「刃先形状をエッジ曲線に合わせる」ボタンを押すと、刃先断面データをエッジ曲線に沿って配置し直します。

6-1. Yリマッピング(線形補間)

刃先形状データのY範囲 $[y_blade_min, y_blade_max]$ をエッジ曲線のY範囲 $[y_edge_min, y_edge_max]$ に線形変換します。

$$y_new = y_edge_min + (y - y_blade_min) / (y_blade_max - y_blade_min) \times (y_edge_max - y_edge_min)$$

6-2. エッジ曲線のZ値補間

y_new に対応するエッジ曲線のZ値を線形補間で求めます。

$$t = (y_new - y_{\{i-1\}}) / (y_i - y_{\{i-1\}})$$

$$z_edge = z_{\{i-1\}} + t \cdot (z_i - z_{\{i-1\}})$$

ここで $y_{\{i-1\}} \leq y_new < y_i$ となるエッジ曲線の区間を使います。

6-3. 接線ベクトルの計算(中央差分)

各Y位置におけるエッジ曲線の接線ベクトルを中央差分で求め、正規化します。

$$\Delta v = (e_{\{i+1\}}.y - e_{\{i-1\}}.y, e_{\{i+1\}}.z - e_{\{i-1\}}.z) \text{ [端点は前進/後退差分]}$$

$$t^{\wedge} = \Delta v / |\Delta v| \text{ (単位接線ベクトル)}$$

※ Three.js の Vector3 として $(x=0, y=\Delta y, z=\Delta z)$ の正規化ベクトルを使います。

6-4. クォータニオン回転

断面の「上向きベクトル」 $(0, 1, 0)$ を接線ベクトル $t^{\wedge}$ へ回転させるクォータニオン q を求め、断面の各点に適用します。

$$q = \text{Quaternion.setFromUnitVectors}((0, 1, 0), t^{\wedge})$$

各断面点 $P = (p.x, 0, p.z)$ (Y成分=0の平面内オフセット) を回転させます。

$$P' = q \cdot P \text{ (クォータニオンによる3D回転)}$$

最終的な3D座標は、アンカー点(刃先位置)に回転済みオフセットを加算した値です。

$$\text{result} = (P'.x, y_new + P'.y, z_edge + P'.z)$$

6-5. クォータニオンの計算方法

2つの単位ベクトル a, b から回転クォータニオンを構築する公式：

$$\text{half} = (a + b) / |a + b|$$

$$q.w = a \cdot \text{half} \text{ (内積)}$$

$q.xyz = a \times half$ (外積)

※ a と b が平行 (同方向) なら $q =$ 恒等クォータニオン、逆向きなら任意の垂直軸まわり 180° 回転を使います。

7. CSV 出力形式

7-1. 刃渡り曲線 CSV(エッジ曲線用・6列)

刃渡り曲線をサンプリングした各点を6列CSVで出力します。

列	値	説明
x	0 (固定)	断面内の横方向位置 (刃先は x=0)
y	xMm [mm]	アゴからの曲線長 (刃渡り方向)
z	-yMm [mm]	画像のY軸を反転した高さ方向
rx	0 (固定)	接線方向の x 成分
ry	dy/ds	接線方向の y 成分 (正規化)
rz	dz/ds	接線方向の z 成分 (正規化)

接線成分 (ry, rz) は前進・後退・中央差分で計算します。

$$ds = \sqrt{(dy^2 + dz^2)}, ry = dy/ds, rz = dz/ds$$

7-2. 合成結果 CSV(3D合成後・6列)

アライメント後の刃先形状全点を同じ6列形式で出力します。

列	値	説明
x	断面横方向 [mm]	V字断面の左右展開量 (刃先=0)
y	刃渡り方向 [mm]	回転・補間後のY位置
z	高さ方向 [mm]	刃渡り曲線Z + 断面Z回転分
rx	0	(現在未使用)
ry	1	(現在未使用)
rz	0	(現在未使用)

7-3. 計測履歴 CSV

計測ごとの刃渡り・刃幅・角度・時刻を CSV 出力します (1計測 = 1行)。

列	内容
#	計測番号
刃渡り(mm)	bladeLength (mm 換算済み、未校正時は px)
刃幅(mm)	包丁幅 (最小外接矩形の短辺)
角度(°)	minAreaRect の傾き angle
時刻	計測時のローカル時刻

以上が本アプリの主要な計算原理です。各ステップは連携しており、カード校正で得たPPMが刃渡りと断面の全計算の基準となります。